Spécifications : Système d'Annotations UML

Bibliothèque de Génération Java vers PlantUML

Documentation Architecturale

18 août 2026

Table des matières

1 Introduction **2**
- 1.1 Comment le code est lu . . . 3
- 1.2 Les trois endroits où quelque chose se décide . . . 3

2 Règles générales et conventions **3**
- 2.1 Convention de nommage . . . 3
- 2.2 Comment un membre est écrit . . . 3
- 2.3 Directions et répétabilité . . . 4
- 2.4 Cibles autorisées . . . 4
- 2.5 Quand deux annotations se rencontrent . . . 4

3 Le Déclencheur et le Seuil : `@UMLClass` **4**

4 La Dérogation d'Exclusion : `@UMLExclude` **5**

5 La Dérogation d'Inclusion : `@UMLInclude` **5**

6 La Précision Métier : `@UMLAssociation` **6**
- 6.1 Les deux notations d'une association . . . 7
- 6.2 Deux attributs qui se font face ne sont qu'une association . . . 7

7 La Classe d'Association : `@UMLAssociationClass` **8**

8 La Multiplicité d'une propriété : `@UMLMultiplicity` **8**

9 Les Éléments Dérivés : `@UMLDerived` **9**

10 Les Contraintes Métier : `@UMLConstraint` **9**

11 Les Liens Transversaux : `@UMLDependency` **9**

12 La Documentation Visuelle : `@UMLNote` **10**

13 Le Regroupement Logique : `@UMLPackage` **10**

14 Les règles du projet : le fichier `umlg.json` 10
14.1 Principe . . . 10
14.2 Le partage avec les annotations . . . 10
14.3 Forme du fichier et vocabulaire . . . 11
14.4 Chaque clé par l'exemple . . . 12
14.5 Le fichier minimal . . . 13
14.6 Un fichier complet . . . 13
14.7 Ce qui est refusé . . . 14

15 L'apparence du diagramme : `UmlStyle` 14

16 Les vues : plusieurs diagrammes pour un projet 14
16.1 Principe . . . 14
16.2 Déclarer une vue . . . 15
16.3 L'ordre d'écriture est l'ordre de lecture . . . 15
16.4 Les vues calculées . . . 15
16.5 Les liens sortants, le passage d'une vue à l'autre, les proportions . . . 15
16.6 Ce que la génération dit d'elle-même . . . 16
16.7 Ce que la génération signale . . . 16
16.8 La sortie : destination, format, génération précédente . . . 16

17 La vérification à la compilation 17
17.1 Principe . . . 17
17.2 Ce qui arrête la compilation . . . 17
17.3 Ce qui produit un avertissement . . . 17
17.4 Faire taire les avertissements . . . 18
17.5 Donner le fichier de règles au compilateur . . . 18

18 Ce qui est hors périmètre 19
18.1 Ce qui relève de l'analyse automatique . . . 19
18.2 Ce qui ne peut pas être déduit . . . 19

19 Mode d'emploi : de zéro à un dossier de diagrammes 19
19.1 1. Déclarer la bibliothèque dans le `pom` . . . 19
19.2 2. Annoter le code . . . 20
19.3 3. Poser `umlg.json` — à un endroit précis, et à un seul . . . 20
19.4 4. Lancer . . . 20
19.5 5. Ce qu'on obtient . . . 21

1 Introduction

Ce document décrit le système d'annotations Java conçu pour la génération automatique de diagrammes PlantUML. L'outil gère nativement les concepts standards — héritage via `extends`, implémentation via `implements`, classes abstraites, énumérations, *records* ; les annotations ci-dessous viennent enrichir et paramétrer cette analyse. Seul le **diagramme de classes** est visé. Le lecteur pressé peut commencer par le mode d'emploi (section 19).

1.1 Comment le code est lu

La bibliothèque travaille **par réflexion sur les classes compilées** : on lui désigne un répertoire de `.class` (typiquement `target/classes`), et non un répertoire de sources. **Ce que la compilation efface est perdu pour le générateur** : les valeurs d'initialisation des attributs et le corps des méthodes — d'où une dépendance qui ne se lit que dans un appel devra être déclarée. Les **signatures déclarées**, elles, survivent à l'effacement de type : un `List<Commande>` dit à lui seul quelle classe l'association vise. Les **noms des paramètres de méthode**, enfin, ne sont écrits dans les `.class` que si le projet est compilé avec `-parameters` (section 2.2).

Une seconde lecture, indépendante, a lieu **à la compilation** : la bibliothèque embarque un processeur d'annotations qui vérifie ce que le développeur écrit, là où il l'écrit (section 17).

1.2 Les trois endroits où quelque chose se décide

— une **annotation** dit une chose sur *cet* élément-ci ;
— le fichier `umlg.json` dit une chose qui vaut pour tout le projet, ou pour une vue (section 14) ;
— l'**appel du générateur** dit ce qui ne touche pas au modèle — l'apparence, la destination (sections 15 et 16.8).

Le partage est étanche : aucune règle ne refait ce qu'une annotation sait faire, et réciproquement (section 14.2). Dans le cas nominal, `@UMLClass` suffit.

2 Règles générales et conventions

2.1 Convention de nommage

Tout paramètre désignant une classe visée par un lien s'appelle `target` (et `source` pour l'extrémité de départ), uniformément sur `@UMLAssociation`, `@UMLAssociationClass` et `@UMLDependency`.

2.2 Comment un membre est écrit

Le générateur emploie la notation d'UML, `nom : Type`, les deux-points entourés d'espaces.

```
class "Vol" {
  - departureTime : Date
  - escales : Aeroport [0..*]
  --
  + reporter(nouvelleHeure : String, minutes : int) : void
  + {static} /compter(code : String) : int
}
```

L'ordre des morceaux est fixe. Pour un attribut : la visibilité, `{static}` s'il y a lieu, le nom (précédé de sa barre oblique s'il est dérivé), le type, la multiplicité entre crochets, puis les contraintes entre accolades. Pour une méthode : la visibilité, `{static}`, le nom, les paramètres entre parenthèses, le type de retour, puis sa multiplicité.

— **Le type de retour** est écrit par défaut, `void` compris ; la règle `"omit": ["return void"]` l'efface pour tout le projet.
— **Les noms des paramètres** ne sont affichés que si le projet a été compilé avec `-parameters` (section 19) ; sinon la signature se réduit à `reporter(String, int) : void`. Un avertissement le **signale** quand aucun paramètre d'aucune méthode dessinée ne porte de nom.

— **L'ordre des membres** est **alphabétique** par défaut, la réflexion ne promettant aucun ordre stable ; `"order": "declaration"` rétablit l'ordre du source. Les littéraux d'une énumération font exception : leur ordre est celui des ordinaux.

— **Devant une multiplicité**, le type écrit est celui de l'**élément**, jamais celui du conteneur : un `Set<String>` marqué `[0..*]` s'écrit `alias : String [0..*]`. Une `Map`, ayant deux paramètres, s'écrit en entier.

— **La multiplicité** n'est écrite que déclarée : par `targetMultiplicity` sous `notation = ATTRIBUTE` (section 6.1), ou par `@UMLMultiplicity` (section 8). Absente, elle vaut *exactement un*.

2.3 Directions et répétabilité

`UMLDirection` (`NONE`, `UP`, `DOWN`, `LEFT`, `RIGHT`) est utilisé partout où une orientation est demandée ; `NONE` est le défaut, PlantUML plaçant alors l'élément lui-même.

`@UMLDependency`, `@UMLNote` et `@UMLConstraint` sont `@Repeatable`. Les autres (`@UMLClass`, `@UMLAssociation`, `@UMLAssociationClass`, `@UMLMultiplicity`, `@UMLPackage`, `@UMLDerived`, `@UMLInclude`, `@UMLExclude`) sont uniques sur leur cible.

```
@UMLDependency(target = ServiceBancaire.class, label = "facture via")
@UMLDependency(target = Journal.class, label = "trace")
@UMLClass
public class Panier { }
```

2.4 Cibles autorisées

Cible	Annotations acceptées
Type (classe, interface, énumération)	`@UMLClass`, `@UMLPackage`, `@UMLAssociationClass`, `@UMLDependency`, `@UMLNote`, `@UMLConstraint`
Attribut	`@UMLInclude`, `@UMLExclude`, `@UMLAssociation`, `@UMLMultiplicity`, `@UMLDerived`, `@UMLConstraint`, `@UMLNote`
Méthode	`@UMLInclude`, `@UMLExclude`, `@UMLMultiplicity`, `@UMLDerived`, `@UMLNote`

Une annotation posée hors de ces cibles est rejetée à la compilation.

2.5 Quand deux annotations se rencontrent

Le détail est réuni en section 17. Trois arbitrages valent d'être connus d'emblée : une **classe visée par un lien sans** `@UMLClass` est une erreur, tandis que les héritages sont ignorés silencieusement — aucune classe n'est créée implicitement ; sur un attribut porteur d'une `@UMLAssociation`, les annotations `@UMLConstraint`, `@UMLDerived` et `@UMLExclude` s'appliquent à l'**association**, c'est-à-dire au trait ou à la ligne ; et une association est **toujours** dessinée, quelle que soit la stratégie de visibilité.

3 Le Déclencheur et le Seuil : `@UMLClass`

Placée sur une classe, interface ou énumération, elle indique que l'entité doit figurer sur le diagramme et fixe un seuil de visibilité par défaut pour ses membres. Elle choisit le périmètre : sans elle, rien n'est dessiné.

— `name` (String) : surcharge le nom affiché.

— `color` (String) : couleur de fond — code hexadécimal (`"#DDEEFF"`, `"#ABC"`, `"#FF000080"` avec transparence) ou nom PlantUML (`"LightBlue"`), vérifié contre le catalogue du moteur qu'énumère `UMLGenerator.knownColors()`.

— `stereotype` (String) : étiquette métier (ex. `"Service"`). Les guillemets « » sont posés par le générateur.

— `strategy` (Enum) : le seuil de visibilité. `PROJECT`, le défaut, s'en remet à la clé `strategy` du fichier `umlg.json`, ou à `NONE` si le projet n'en déclare aucune ; `NONE` donne une coquille vide, sauf les membres explicitement `@UMLInclude` ; `PUBLIC`, `PROTECTED`, `PACKAGE` et `PRIVATE` retiennent respectivement les membres `public` ; `public` et `protected` ; `public` et sans modificateur ; tous.

Les composants d'un record échappent à la stratégie et se dessinent même sous `NONE`. `@UMLExclude` garde le dernier mot ; un champ *ajouté* au record (forcément statique) reste un membre ordinaire.

Rien n'est masqué sans demande. Le tri se déclare dans `umlg.json` (section 14). Seul est écarté d'office ce que le compilateur a fabriqué — attributs synthétiques, référence à la classe englobante. Les constructeurs ne sont jamais lus, et aucune annotation ne peut en faire apparaître un.

Ci-dessous, le projet a écrit `"hide": ["get*"]` dans son `umlg.json` ; sans cette entrée, `getPseudo()` paraîtrait comme n'importe quelle autre méthode publique.

```
// Affiche les membres que la strategie autorise, moins ce que le
// fichier de regles ecarte.
@UMLClass(stereotype = "Entity", strategy = UMLVisibility.PACKAGE)
public class Utilisateur {
    public String pseudo;        // Sera affiche
    String email;                // Sera affiche (package-private)
    protected int age;           // Sera ignore (hors strategie)
    private String motDePasse;   // Sera ignore (hors strategie)

    public Utilisateur() {}      // Sera ignore (jamais de constructeur)
    public String getPseudo() {}// Sera ignore (motif "get*" de hide)

    public void valider() {}     // Sera affiche (methode metier publique)
}
```

4 La Dérogation d'Exclusion : `@UMLExclude`

Force la dissimulation d'un attribut ou d'une méthode que la `strategy` autorisait à paraître — loggers, constantes internes. **Aucun paramètre.**

Posée sur la déclaration d'une méthode — dans une interface, ou dans une classe parente —, elle vaut pour **chacune de ses redéfinitions**. Une méthode statique ou privée, qui ne redéfinit rien, ne regarde qu'elle-même.

```
@UMLClass(strategy = UMLVisibility.PRIVATE)
public class ConnexionBase {
    private String url; // Affiche car strategie globale sur PRIVATE

    @UMLExclude
    private Logger log; // Ignore explicitement pour alleger le graphe
}
```

5 La Dérogation d'Inclusion : `@UMLInclude`

Force l'apparition d'un membre que son niveau d'encapsulation aurait masqué, et peut de surcroît le **renommer**.

— `name` (String) : le nom sous lequel le membre paraît, à la place du sien. La chaîne vide, le défaut, veut dire « le nom du code ». Les notes suivent le nom affiché.

— `asAttribute` (boolean, `false` par défaut) : écrit la méthode en **notation attribut** — `xKeycodeByXKeyName : Map<String, Short>`, sans parenthèses, dans le compartiment des attributs. Exige une méthode **sans paramètre** qui **rend quelque chose** ; la ligne garde son soulignement si la méthode est statique, sa barre oblique si elle est dérivée, sa multiplicité si elle en porte une.

Sur un composant de record, une seule ligne. Le compilateur recopie l'annotation sur le champ **et** sur l'accesseur ; seul le composant est retenu. Une `@UMLInclude` posée sur un accesseur *écrit à la main* reste, elle, affichée.

```
@UMLClass(strategy = UMLVisibility.PUBLIC)
public class MoteurDeRecherche {

    public void chercher() {} // Affiche car public

    private int compteur; // Ignore car private

    @UMLInclude
    private void algorithmeSecret() {} // Force l'affichage grace a l'
   annotation

    @UMLInclude(name = "xKeycodeByXKeyName")
    public Map<String, Short> codeByName() { ... }
    // Affiche : xKeycodeByXKeyName() : Map<String, Short>
}
```

6 La Précision Métier : `@UMLAssociation`

Précise la sémantique UML d'une relation portée par un attribut : type de lien, multiplicités, rôles, classe ciblée. Le code `List<Commande> commandes` indique qu'il y a plusieurs commandes ; seul le métier sait si c'est `0..*` ou `1..*`, et s'il s'agit d'une association ou d'une composition.

— `target` (Class) : la classe cible. **Facultatif**, le type de l'attribut la nommant déjà (section 1.1) ; nécessaire quand le type ne dit rien — collection sans paramètre, `Map` dont on ne saurait quelle moitié est visée, classe absente du type.

— `notation` (Enum) : `ARROW` (défaut) ou `ATTRIBUTE` — la **forme** du dessin (section 6.1).

— `type` (Enum) : `ASSOCIATION`, `COMPOSITION`, `AGGREGATION`.

— `sourceMultiplicity`, `targetMultiplicity` (String) : multiplicités (ex. `"1"`, `"0..1"`, `"0..*"`) ; `sourceRole`, `targetRole` (String) : noms des rôles aux extrémités ; `label` (String) : texte sur la flèche ; `direction` (Enum) : `UMLDirection`, qui contraint le placement de *ce* trait.

L'ordre n'est pas un paramètre : il s'écrit `@UMLConstraint(rule = "ordered")`, comme n'importe quelle autre règle formelle.

```
@UMLConstraint(rule = "ordered")
@UMLAssociation(target = Commande.class, targetMultiplicity = "0..*")
private List<Commande> commandes;   // "achats 0..* {ordered}"
```

```
public class Utilisateur {
    @UMLConstraint(rule = "ordered")
    @UMLAssociation(
        target = Commande.class,
        type = UMLRelationType.COMPOSITION,
        sourceMultiplicity = "1",
        targetMultiplicity = "0..*",
        targetRole = "achats",
```

```
        label = "passe"
    )
    private List<Commande> commandes;
}
```

6.1 Les deux notations d'une association

UML n'autorise **qu'une notation à la fois** : un trait entre deux boîtes, ou une ligne dans la boîte de départ. Sous `ARROW` l'attribut est retiré de la boîte au profit du trait ; sous `ATTRIBUTE` il y reste et aucun trait n'est tracé.

Valeur	Ce qui est dessiné	Quand la choisir
`ARROW` (défaut)	Un trait décoré à ses extrémités : `Client "1" *-- "achats 0..*" Commande`	Les associations qui structurent le domaine, celles que le lecteur doit voir en premier.
`ATTRIBUTE`	Une ligne du compartiment des attributs : `- commandes: Commande [0..*]`	Quand une classe en cite dix autres, ou quand la cible n'a rien d'intéressant à montrer.

```
@UMLClass
public class Client {
    // Un trait vers Commande (comportement par defaut).
    @UMLAssociation(target = Commande.class, targetMultiplicity = "1")
    private Commande courante;

    // Une ligne dans la boite : "- archivees: Commande [0..*]"
    @UMLAssociation(target = Commande.class, targetMultiplicity = "0..*",
                    notation = UMLNotation.ATTRIBUTE)
    private List<Commande> archivees;
}
```

Ce que la notation `ATTRIBUTE` retient. Trois paramètres : `target` donne le **type** affiché (la classe visée, et non `List`) ; `targetMultiplicity` la **multiplicité** entre crochets ; `targetRole` le **nom** de la propriété, à défaut celui de l'attribut Java. S'y ajoutent la visibilité et `{static}`, la barre oblique de `@UMLDerived`, puis les `@UMLConstraint`. Les paramètres qui décorent un *trait* — `type`, `sourceMultiplicity`, `sourceRole`, `label`, `direction` — ne sont pas dessinés, et le compilateur les nomme un à un. La classe visée doit porter `@UMLClass` dans les deux notations.

```
@UMLConstraint(rule = "ordered")
@UMLConstraint(rule = "non vide")
@UMLDerived
@UMLAssociation(target = Commande.class, targetMultiplicity = "1..*",
                notation = UMLNotation.ATTRIBUTE)
protected List<Commande> achats;
// Ligne produite : # /achats: Commande [1..*] {ordered} {non vide}
```

6.2 Deux attributs qui se font face ne sont qu'une association

`Eleve` cite `Cours`, et `Cours` cite `Eleve` : UML n'y voit qu'**une** association, dont chaque extrémité est nommée par la classe d'en face. Traduits séparément, les deux attributs donneraient deux traits parallèles :

```
Eleve --> "cours 0..*" Cours        % ce que disait Eleve
Cours --> "inscrits 1..*" Eleve     % ce que disait Cours
```

La paire est fusionnée au rendu, et le dessin porte le trait unique qu'UML décrit :

```
Eleve "inscrits 1..*" --> "cours 0..*" Cours
```

Elle n'a lieu que si le second lien **n'ajoute rien qu'un rôle et une multiplicité** — une extrémité source ne sait pas porter davantage — et si le premier n'a pas écrit son côté source ; sinon les deux traits restent, et un avertissement les nomme.

7 La Classe d'Association : `@UMLAssociationClass`

Identifie une classe comme porteuse des attributs d'une relation entre deux autres (table de jointure enrichie) : une ligne en pointillés relie le lien principal à la classe.

Paramètres : `source` et `target` (Class), les deux extrémités, toutes deux **obligatoires**. Les déclarer suffit à tracer le lien ; y ajouter une `@UMLAssociation` entre `source` et `target` dessinerait un second trait. Les deux extrémités doivent porter `@UMLClass`, et la classe annotée aussi ; aucune extrémité ne peut être la classe d'association elle-même, mais toutes deux peuvent désigner *la même* classe.

```
@UMLAssociationClass(source = Etudiant.class, target = Cours.class)
@UMLClass
public class Inscription {
    private LocalDate dateInscription;
    private double noteObtenue;
}
```

8 La Multiplicité d'une propriété : `@UMLMultiplicity`

Déclare combien d'objets une propriété tient, entre crochets à côté de son nom : sur un attribut, le nombre d'objets tenus ; sur une méthode, le nombre d'objets rendus — c'est le seul moyen de qualifier une méthode.

```
@UMLClass(strategy = UMLVisibility.PRIVATE)
public class CanonicalMnemonic {
    @UMLMultiplicity("0..*")
    private String deprecatedAliases;   // - deprecatedAliases: String
   [0..*]

    @UMLMultiplicity("0..1")
    private int ucp;                    // - ucp: int [0..1]

    @UMLMultiplicity("0..*")
    public String names() { ... }       // + names(): String [0..*]
}
```

`value` (String) : la multiplicité, dans le vocabulaire d'UML — `"1"`, `"0..1"`, `"*"`, `"1..*"`, `"2..5"` ; le nom `value` permet la forme courte `@UMLMultiplicity("0..*")`. Elle vise les propriétés qui n'ont **aucune classe à l'autre bout** : la multiplicité d'un *lien* se déclare par `targetMultiplicity`, et cumuler les deux est une erreur. Avec `@UMLDerived` et `@UMLConstraint` la combinaison est libre : `- /etiquettes: String [1..*] {sans doublon}`.

Ce que le projet peut traduire à sa place. La règle `"multiplicity": "brackets"` transcrit ce que le type dit déjà : `Optional<X>` vaut « zéro ou un », `Set<X>` « zéro ou plusieurs ». Elle ne dit jamais *plus* que le type — `0..*` et non `1..*` — et l'annotation l'emporte sur elle. Un type enveloppe redevient alors primitif : `Optional<Integer>` s'écrit `int [0..1]`.

9 Les Éléments Dérivés : `@UMLDerived`

Indique qu'un attribut, une méthode ou une association est dérivé — calculé, non stocké. UML le note d'un slash (/), que l'annotation demande au rendu puisque Java ne l'accepte pas dans un identifiant. Elle ne s'applique pas à une classe. **Aucun paramètre.**

La barre oblique appartient au **nom** de la propriété, et à lui seul : sur un lien dessiné en trait elle précède le rôle cible, dans une boîte le nom de la ligne.

```
@UMLClass(strategy = UMLVisibility.PRIVATE)
public class Personne {
    private LocalDate dateNaissance;

    @UMLDerived
    private int age; // Affiche en UML sous la forme : - /age: int

    @UMLDerived
    @UMLAssociation(target = Commande.class, targetRole = "encours",
                    targetMultiplicity = "0..*")
    private List<Commande> commandesEnCours; // Trait portant "/encours
    0..*"

    @UMLDerived
    @UMLAssociation(target = Commande.class, targetMultiplicity = "0..*",
                    notation = UMLNotation.ATTRIBUTE)
    private List<Commande> commandesSoldees; // - /commandesSoldees:
    Commande [0..*]
}
```

10 Les Contraintes Métier : `@UMLConstraint`

Ajoute une règle formelle, écrite entre accolades `{ }`, sur une classe, un attribut ou un lien (`{subset}`, « solde positif »). `rule` (String) : la règle, **écrite nue**, les accolades étant ajoutées par le générateur. Annotation **répétable**. Trois positions :

- **sur un attribut** : à la suite de sa ligne — `- solde: double {toujours positif}` ;
- **sur un attribut porteur de `@UMLAssociation`** : là où l'association est dessinée — à l'extrémité cible du trait sous `ARROW`, à la fin de la ligne sous `ATTRIBUTE` —, après la multiplicité ;
- **sur une classe** : dans une bulle accolée à la boîte.

```
public class CompteBancaire {

    @UMLConstraint(rule = "solde >= decouverAutorise")
    private double solde; // Affichera {solde >= decouverAutorise} a cote
    de l'attribut
}
```

11 Les Liens Transversaux : `@UMLDependency`

Déclare une dépendance vers une classe qui n'est pas un attribut — utilisée localement dans une méthode. C'est la seule façon de révéler qu'une classe en utilise une autre sans en stocker l'état, le corps des méthodes ayant disparu à la compilation. Paramètres : `target` (Class) et `label` (String), la raison du lien. Annotation **répétable**.

```
@UMLDependency(target = ServiceBancaire.class, label = "facture via")
public class Panier {
    public void valider() {
```

```
        ServiceBancaire sb = new ServiceBancaire();
        sb.payer();
    }
}
```

12 La Documentation Visuelle : `@UMLNote`

Génère une bulle de commentaire rattachée à une classe, un attribut ou une méthode. Annotation **répétable**. `text` (String) : le contenu, obligatoire et non vide. `direction` (Enum) : `UMLDirection` — `NONE`, le défaut, place la bulle **à droite** ; `UP` et `DOWN` au-dessus et en dessous.

Ce à quoi une note peut s'accrocher. Une bulle ne se dessine qu'attachée à un élément présent. Deux cas la laissent sans support, et le compilateur avertit : le **membre visé n'est pas affiché** ; ou l'**attribut visé porte une `@UMLAssociation` dessinée en trait** — les notes rattachées à un lien sont hors périmètre, et poser la note sur la classe est alors la seule façon d'en parler. Sous `ATTRIBUTE`, la note s'accroche à la ligne, en visant le **nom affiché**.

```
public class MoteurDeRecherche {
    @UMLNote(text = "Algorithme critique, ne pas modifier", direction =
    UMLDirection.RIGHT)
    public void indexer() {
    }
}
```

13 Le Regroupement Logique : `@UMLPackage`

Regroupe visuellement des classes dans un rectangle nommé. `name` (String), **obligatoire**, fait l'identité du groupe : deux classes déclarant le même `name` entrent dans le même rectangle. `color` (String) : couleur de fond, écrite comme celle d'une boîte.

Le groupe est déclaré, jamais déduit du paquetage Java : deux classes de paquetages différents peuvent figurer dans le même rectangle, et inversement. Il ne change rien d'autre — les boîtes sont écrites à l'identique et les traits continuent de le traverser.

```
@UMLPackage(name = "Module Paiement", color = "#EEEEEE")
@UMLClass
public class Facture {
}
```

14 Les règles du projet : le fichier `umlg.json`

14.1 Principe

Ce qui vaut pour le projet entier — « dans ce diagramme, un `ImmutableList` s'affiche `List` » — se déclare dans un fichier facultatif, `umlg.json`, posé **à la racine du répertoire analysé** ; pour un projet Maven ordinaire, `src/main/resources/umlg.json` (section 19.3).

14.2 Le partage avec les annotations

Le critère est unique :

Cette décision désigne-t-elle un élément par son nom ?

Si oui, c'est une annotation ; si non, c'est une règle. D'où : aucune règle ne désigne un membre nommément (`"hide": ["Representation.string"]` n'existe pas), n'en range un

dans un rectangle (pas de clé `group`), ni ne fabrique une boîte ou un lien. Et une priorité sans exception : **l'annotation l'emporte toujours sur la règle** — `@UMLInclude` ramène ce qu'un `hide` écarte, `@UMLMultiplicity` l'emporte sur `"multiplicity": "brackets"`, `@UMLClass(strategy)` sur `strategy`.

Ce qu'on veut	Où cela s'écrit
Renommer une boîte	`@UMLClass(name = ...)`
Renommer un membre	`@UMLInclude(name = ...)`
Taire un membre précis	`@UMLExclude`
Ranger des classes dans un rectangle	`@UMLPackage(name = ...)`

14.3 Forme du fichier et vocabulaire

Un **objet JSON**, dont chaque clé dit une nature de règle, encodé en **UTF-8**. Le vocabulaire est **fermé** : une clé inconnue arrête la génération. Un fichier vide, ou réduit à `{}`, ne règle rien ; écrire le défaut est permis (`"order": "alphabetical"`). Onze clés, et **aucune** qui refasse ce qu'une annotation sait faire. Dans le tableau, ce qui est en `chasse fixe` s'écrit tel quel, ce qui est en *<italique entre chevrons>* est à remplacer, et une barre verticale sépare des valeurs au choix.

Forme de la règle	Ce qu'elle fait
`"hide": [`*<motif>*`, `*<motif>*`...]`	Écarte les **membres** — méthodes comme attributs — dont le nom suit l'un des motifs, où `*` remplace n'importe quelle suite de caractères. Ex. : `["get*", "log", "*Cache"]`
`"omit": [`*<"visibility" | "parameter names" | "return void" | "abstract">*`, ...]`	Retire un morceau de la ligne : les symboles `+ - # ~` ; les noms des paramètres (`from(KeyboardMap)`) ; le type rendu `void` ; la marque `{abstract}` — le mot-clé de la boîte, lui, reste écrit.
`"strategy":` *<"none" | "public" | "protected" | "package" | "private">*	La stratégie des classes qui ne déclarent pas la leur.
`"types": {` *<type du code>*`:` *<nom affiché>*`, ... }`	Renomme *n'importe quel* type dans les boîtes, chevrons compris.
`"order":` *<"declaration" | "alphabetical">*	L'ordre des membres ; `"alphabetical"` est le défaut.
`"multiplicity": "brackets"`	`Set<X>` s'écrit `X [0..*]`, `Optional<X> X [0..1]`.
`"title":` *<texte libre>*	Titre de la vue intégrale, pris tel quel.
`"direction":` *<"top-to-bottom" | "left-to-right">*	Son sens de lecture.
`"showEmptyMembers": true`	Rétablit les compartiments vides.
`"views": [ { "name":` *<nom>*`,` `"retain": [`*<sélecteur>*`, ...] } ]`	Déclare une vue et ce qu'elle retient, dans l'ordre. Deux clés facultatives dans le même objet : `"remove"`, ce qu'elle retire de ce qu'elle a retenu, et `"direction"`, son sens de lecture.
`"viewsByPackage": true`	Une vue par paquetage, sans rien déclarer.

Un *<sélecteur>* s'écrit de trois façons, et de trois seulement : une **chaîne** pour une classe, par son nom simple ou complet ; `{ "package":` *<paquetage>* `}` ; `{ "around":` *<Classe>* `}` (section 16). Un *<motif>* et un *<nom de type>* ne sont contraints par aucune liste : ce sont les noms du projet analysé.

Les commentaires. JSON n'en a pas : une clé dont le nom commence par `_` n'est pas une règle et n'est **pas lue**, dans l'objet de tête comme dans une vue.

Le schéma, et l'aide de l'éditeur. Une douzième clé, `"$schema"`, n'est pas une règle non plus : elle n'est pas lue, et sert à désigner le **schéma JSON** livré avec la bibliothèque (`umlg-schema.json`). L'éditeur qui la voit complète les clés, propose les valeurs des énumérations et souligne une faute avant même la génération — ce que le refus à la lecture ne peut faire qu'après coup.

```
{
  "$schema": "https://raw.githubusercontent.com/CharlesBaudoux/
   STAGE_UML_JAVA/main/UMLGenerator/src/main/resources/umlg-schema.json",
  "strategy": "public"
}
```

```
{
  "_": "Les types de Guava n'apprennent rien au lecteur du diagramme.",
  "_pourquoi": "Une seconde note ne gene pas la premiere.",
  "types": { "ImmutableList": "List" }
}
```

14.4 Chaque clé par l'exemple

`strategy`. La stratégie des classes dont le `strategy` vaut `PROJECT`, c'est-à-dire le défaut de `@UMLClass`. Sans cette clé, aucun membre n'est dessiné et les boîtes sortent vides — ce que la génération **signale** ; un `"none"` **écrit** donne le même dessin sans le signalement.

```
"strategy": "public"
"_autres": "none, protected, package, private"
```

`hide`. Un tableau de **motifs de noms**, un par entrée. L'étoile remplace n'importe quelle suite de caractères, y compris aucune ; tout le reste est pris au pied de la lettre, et la casse compte.

```
"hide": ["get*", "set*", "is*", "toString", "hashCode",
         "with*", "visit*", "*Cache"]
```

Un motif ne regarde que le **nom**, jamais la signature : `"get*"` emporte aussi un `getPrix(Client)` métier, que `@UMLInclude` ramène. Il vise **méthodes et attributs indifféremment**, et l'étoile s'écrit : `"get"` ne masque que le membre nommé exactement `get`. Un motif qui n'a rien masqué est signalé en fin de génération.

`omit`. Ces réglages ne font disparaître aucun membre : ils retirent **un morceau de la ligne** d'un membre qui reste affiché, et aucun `@UMLInclude` ne les contredit. `"return void"` ne vaut que pour le retour, et que pour `void` : un type de retour utile n'est jamais effacé.

```
"_omit": "visibility     : 'valider()' et non '+ valider()'
          parameter names: 'from(KeyboardMap)' et non 'from(m : KeyboardMap
   )'
          return void    : 'valider()' et non 'valider() : void'",
"omit": ["visibility", "parameter names", "return void"]
```

`types`. Remplace un nom de type par un autre, **partout où ce type paraît dans une boîte** : attributs, types de retour, paramètres, chevrons compris. Le nom du code étant une **clé**, deux règles contradictoires sur le même type sont refusées par l'analyseur lui-même. Il s'écrit au choix simple ou complet, le nom complet l'emportant — de quoi viser une classe précise quand deux paquetages en ont une homonyme. La règle ne touche pas le **nom d'une boîte**, qui relève de `@UMLClass(name)`.

```
"_types": "'ImmutableList<ImmutableSet<String>>' s'ecrit 'List<Set<String
   >>'",
```

```
"types": {
  "ImmutableList": "List",
  "com.google.common.collect.ImmutableSet": "Set"
}
```

```
"_order": "L'ordre du source ; le defaut est 'alphabetical'.",
"order": "declaration",

"_multiplicity": "Set<String> aliases   ->  aliases : String [0..*]
                  Optional<Integer> ucp ->  ucp : int [0..1]",
"multiplicity": "brackets"
```

`title`, `direction`, `showEmptyMembers`. Le titre est pris tel quel, espaces compris. Sans stratégie déclarée une boîte n'a aucun membre, et rétablir les compartiments vides dessinerait deux rectangles vides sous chaque nom.

```
"title": "Architecture du module Facturation",
"direction": "left-to-right",
"_direction": "aussi : top-to-bottom, le defaut",
"showEmptyMembers": true
```

`views` et `viewsByPackage` : détaillés en section 16.

14.5 Le fichier minimal

Trois clés suffisent au cas nominal :

```
{
  "strategy": "public",
  "hide": ["get*", "set*"],
  "types": { "ImmutableList": "List" }
}
```

14.6 Un fichier complet

```
{
  "_dessin": "---- Le dessin d'ensemble ----",
  "title": "Architecture du module Facturation",
  "direction": "left-to-right",

  "_boites": "---- Ce que les boites montrent ----",
  "strategy": "public",
  "hide": ["get*", "set*", "is*", "toString", "hashCode", "equals",
           "with*", "visit*", "log", "LOGGER"],
  "omit": ["return void", "visibility", "parameter names"],
  "order": "declaration",
  "multiplicity": "brackets",
  "types": { "ImmutableList": "List" },

  "_vues": "---- Les vues ----",
  "views": [
    { "name": "Le domaine",
      "retain": [{ "package": "com.exemple.modele" }] },
    { "name": "Facturation",
      "retain": ["Facture", "LigneFacture", "Client"],
      "direction": "left-to-right" },
    { "name": "Autour de Facture",
      "retain": [{ "around": "Facture" }] }
  ],
  "viewsByPackage": true
```

```
}
```

14.7 Ce qui est refusé

Chaque refus nomme le fichier, le **chemin fautif** — `views[1].direction`, `omit[0]` — et la forme attendue, et **arrête la génération** plutôt que d'appliquer le fichier à moitié ; les fautes de syntaxe gardent le numéro de ligne. Sont refusés : un JSON mal formé (virgule finale, accolade oubliée) ou un fichier qui n'est pas en UTF-8 — un `#` de commentaire échoue ici, une note s'écrivant dans une clé `_` ; un fichier commençant par une **marque d'ordre des octets** (BOM), qu'un éditeur pose sans le dire et qu'aucun regard ne rattrape, refusé par un message qui la nomme ; une valeur qui n'est pas de la sorte attendue, `"hide": "get*"` au lieu d'un tableau ; une clé inconnue (`"couleur"`), une valeur inconnue d'un `omit`, `order`, `direction` ou `multiplicity`, une stratégie inconnue ou `"strategy": "project"` ; une clé écrite deux fois, ou deux fois la même valeur dans `omit` ; un `title` vide, un motif vide ou réduit à `*`, une moitié vide d'un `types` ; un motif contenant un point (`"Representation.string"`), pour lequel `@UMLExclude` est fait ; une espace au milieu d'un motif, d'un nom de type ou d'un sélecteur ; une vue sans `name` ou sans `retain`, un `retain` vide, une vue déclarée deux fois, une clé inconnue dans une vue ; un sélecteur ne désignant aucune classe.

Une entrée syntaxiquement correcte dont le nom ne correspond à rien — `"ImutableList": "List"`, le motif `"LOGGR"`, l'étoile oubliée d'un `"get"` — n'est pas refusée : la génération relève en fin de scan les entrées restées sans effet et les signale (section 16.7).

15 L'apparence du diagramme : `UmlStyle`

L'habillage — couleurs des boîtes, épaisseur et teinte des traits, police — se déclare **à l'appel du générateur**, par `.style(UmlStyle.UMLG)` sur le builder (section 16.8). Deux valeurs, et aucun accès aux `skinparam`. Deux réglages y échappent : les visibilités sont écrites `+`, `-`, `#`, `~` plutôt qu'en pastilles colorées (`classAttributeIconSize 0`), et l'**espacement** est calculé sur la plus longue étiquette du diagramme.

Valeur	Ce qui est dessiné	Quand la choisir
`PLANTUML` (défaut)	Boîtes grises, pastille `C` ou `I`, traits par défaut.	Le cas nominal : le rendu que l'utilisateur reconnaît.
`UMLG`	Bandeau bleu pâle sur fond blanc cassé, bords bleu foncé, coins arrondis, pas d'ombre, nom des liens en italique coloré, pastille masquée.	À l'impression, ou sur un fond clair.

16 Les vues : plusieurs diagrammes pour un projet

16.1 Principe

La bibliothèque produit, en une exécution, **un dossier de diagrammes** plutôt qu'une image :

```
diagrammes/
|-- projet.svg          <- la vue integrale, toujours produite
`-- vues/
    |-- le-domaine.svg
    |-- autour-de-compte.svg
    `-- com.banque.modele.svg
```

La **vue intégrale** est toujours écrite à la racine, et porte le titre et le sens de lecture déclarés par `title` et `direction`. Les autres vivent dans un sous-dossier ; chacune porte son propre titre — son nom — et son propre sens de lecture.

16.2 Déclarer une vue

Une vue tient dans un objet : son nom, ce qu'elle retient dans l'ordre où on veut le lire, ce qu'elle retire, son sens de lecture.

```
"views": [
  { "name": "Le domaine",
    "retain": [{ "package": "com.banque.modele" }] },
  { "name": "Facturation",
    "retain": ["Facture", "LigneFacture", "Client"],
    "direction": "left-to-right" },
  { "name": "Autour de Compte",
    "retain": [{ "around": "Compte" }] },
  { "name": "Le domaine sans les lecteurs",
    "retain": [{ "package": "com.banque" }],
    "remove": ["XkbSymbolsReader"] }
]
```

Trois façons de désigner des classes, et trois seulement :

Sélecteur	Ce qu'il retient
`"Facture"`	Une classe, par son nom simple ou complet.
`{ "package": "com.exemple" }`	Toutes les classes du paquetage, correspondance exacte — un sous-paquetage n'entre pas.
`{ "around": "Facture" }`	La classe et ses voisines immédiates ; le rayon est de un.

Un voisinage peut être vide. Les voisins d'`around` sont les extrémités des **relations déclarées** — `@UMLAssociation`, `@UMLDependency`, héritages entre classes annotées — et non les types que le code manipule : une classe très entourée peut n'avoir aucun voisin déclaré, et la vue se réduit alors à une boîte seule, ce dont la génération **avertit**. Un sélecteur qui ne désigne *aucune* classe, lui, arrête la génération.

16.3 L'ordre d'écriture est l'ordre de lecture

Les classes d'une vue sont dessinées dans l'ordre où elles sont écrites : le moteur place déjà ce qui est relié, mais ne peut pas deviner l'ordre de deux boîtes que *rien* ne relie. La bibliothèque enchaîne donc par un lien invisible les **blocs** de la vue — un bloc étant un groupe de boîtes que les traits relient, de proche en proche — dans l'ordre où leur première boîte est écrite. Un sélecteur `"package"` ou une vue calculée ne portent aucun ordre : leurs boîtes sont rangées par nom complet.

16.4 Les vues calculées

```
"viewsByPackage": true
```

Une vue par paquetage Java, sans rien déclarer — mais retenant *tout* le paquetage, là où un auteur en écarterait les classes techniques.

16.5 Les liens sortants, le passage d'une vue à l'autre, les proportions

Une relation dont l'autre bout n'est pas dans la vue **disparaît, sans un mot** : la boîte manquante n'est jamais ajoutée pour sauver le trait. Chaque boîte porte en revanche un **lien**

cliquable vers la première vue déclarée qui la retient ; une vue ne renvoie jamais vers elle-même, et les chemins sont relatifs, si bien que le dossier reste déplaçable.

Une vue dont les boîtes s'alignent sur une seule rangée donne un ruban illisible. Deux garde-fous : **l'enchaînement des blocs s'arrête à trois** ; et les **proportions sont mesurées** — la bibliothèque relit les dimensions de ce qu'elle vient d'écrire et **avertit** lorsque le rapport dépasse quatre pour un, sans rien corriger.

16.6 Ce que la génération dit d'elle-même

```
12 diagram(s) written to /chemin/diagrammes
  21 class(es) drawn, 23 ignored (no @UMLClass)
```

Sous une douzaine, les classes ignorées sont nommées ; au-delà, la génération écrit `ignored-classes.txt` à la racine de la sortie. Le compte rendu est **rendu, non imprimé** : `generate()` rend le `GenerationReport`, que la ligne de commande écrit sur la sortie standard et le plugin Maven passe à son log — les avertissements de même. Chaque image est enfin accompagnée de sa **source PlantUML**, sous le même nom et l'extension `.puml`.

16.7 Ce que la génération signale

Un **avertissement** dit que le diagramme est *exact*, mais qu'il montre moins que ce que son auteur croit avoir demandé ; chacun se tait dès qu'un choix explicite a été écrit.

Situation	Ce que l'auteur découvrirait sinon sur l'image
Des boîtes sans membre, aucune `strategy` déclarée nulle part	Le défaut ne dessine aucun membre. Le message nomme les boîtes et les trois remèdes ; il se tait devant un `"strategy": "none"` **écrit**.
Aucun paramètre d'aucune méthode ne porte de nom	Projet compilé sans `-parameters` (section 2.2).
Des `.class` que le chargeur n'a pas su lier	Dépendance absente du classpath, ou compilation partielle. Ces classes sont écartées et nommées ; rien ne dit si elles portaient `@UMLClass`.
Deux boîtes portant le même nom affiché	Deux classes homonymes : les traits aboutissent à la bonne boîte, mais le lecteur voit deux rectangles identiques.
Une vue `"around"` sans voisin	Les voisins viennent des relations déclarées, pas des types du code.
Une règle de nommage restée lettre morte	Une entrée de `types` ou un motif de `hide` qui n'a rien touché : faute de frappe, ou étoile oubliée.
Une vue aux proportions illisibles	Le ruban décrit ci-dessus.

Ce qui arrête la génération. Une annotation contradictoire, une couleur inconnue de PlantUML, un fichier de règles fautif, une vue nommant une classe absente, un répertoire où aucune classe ne porte `@UMLClass` — et **Graphviz introuvable** : PlantUML dessinerait alors une image portant « Cannot find Graphviz » que rien ne distingue d'un dessin réussi, aussi le moteur est-il vérifié avant de dessiner.

16.8 La sortie : destination, format, génération précédente

```
UMLGenerator.builder()
    .sourceDirectory("target/classes")
    .outputDirectory("diagrammes")  // toutes les vues ; outputFile n'ecrit
    .outputFormat(".svg")           //   que le diagramme d'ensemble
    .keepPrevious(true)             // facultatif ; false par defaut
    .style(UmlStyle.UMLG)           // facultatif ; PLANTUML par defaut
```

```
    .build()
    .generate();
```

Par défaut, les diagrammes précédents sont effacés, de sorte que le dossier décrive l'état courant du code ; seuls les fichiers que la bibliothèque écrit sont retirés. Sous `keepPrevious`, la génération va dans un dossier voisin numéroté — `diagrammes-2`, `diagrammes-3`.

```
UMLGenerator <repertoire-analyse> [fichier.svg|dossier] [png|svg]
```

Un second argument finissant par `.svg` ou `.png` demande ce seul fichier ; tout autre nom demande un dossier de vues.

Depuis Maven , la forme à préférer : le plugin `umlg-maven-plugin` et son goal `generate` (section 19), qui transmet en outre le **classpath d'exécution** du projet — charger une classe résout son parent, lire un attribut résout son type.

17 La vérification à la compilation

17.1 Principe

La bibliothèque embarque un **processeur d'annotations** (JSR 269) qui s'exécute à l'intérieur de `javac` : il n'engendre aucun fichier, il lit les annotations et pointe la valeur en cause dans le source. Il construit les objets du modèle avec les valeurs lues et traduit en erreur de compilation l'exception que le modèle lève déjà ; le message de `javac` est donc mot pour mot celui que la génération aurait produit. Deux gravités :

— **Erreur** : *ce code ne décrit aucun diagramme.* La compilation s'arrête.
— **Avertissement** : *ce code décrit un diagramme, mais vraisemblablement pas celui que son auteur croit.* La compilation continue — l'annotation fait toujours foi.

17.2 Ce qui arrête la compilation

— cible d'un lien dépourvue de `@UMLClass`, cible primitive (`int.class`) ou qui n'est pas une classe ;
— annotation de lien posée sur une classe dépourvue de `@UMLClass` ;
— multiplicité illisible (`"beaucoup"`) ou inversée (`"3..1"`) ;
— couleur inconnue de PlantUML ;
— texte fait uniquement d'espaces (nom, rôle, label, stéréotype, titre), ou `@UMLPackage(name)` vide ;
— `@UMLInclude(name)` et `targetRole` sur la même ligne ;
— `@UMLInclude` et `@UMLExclude` sur le même membre ;
— `@UMLMultiplicity` et `@UMLAssociation` sur le même attribut ;
— `asAttribute = true` sur une méthode à paramètres, sur une méthode `void`, ou sur un attribut ;
— `@UMLAssociationClass` dont une extrémité est la classe annotée ;
— dépendance d'une classe vers elle-même, ou deux fois la même.

17.3 Ce qui produit un avertissement

Situation	Ce que l'auteur attendait, et n'aura pas
`@UMLInclude` sur un membre déjà affiché, `@UMLExclude` sur un membre déjà masqué, `@UMLInclude(name)` reprenant le nom du code	La dérogation ne déroge à rien, le renommage n'a pas eu lieu. Un `name` qui renomme vraiment n'est jamais signalé.
`@UMLInclude` ou `@UMLExclude` sur un attribut porteur de `@UMLAssociation`	Sous `ARROW` l'attribut est un lien — rien ne l'y ramène, et l'exclure supprime le **trait** ; sous `ATTRIBUTE` il est déjà affiché, et c'est la **ligne** qui disparaît.
Paramètre de trait écrit sous `notation = ATTRIBUTE` (`type`, `label`, `direction`, `sourceRole`, `sourceMultiplicity`)	Ces valeurs ne seront dessinées nulle part ; le message les nomme une à une.
`stereotype` contenant déjà `« »`, ou `rule` portant déjà ses accolades	Elles seraient dessinées en double.
Deux classes affichant la même étiquette, ou deux couleurs pour un même groupe	Deux rectangles que le lecteur ne peut pas distinguer ; pour la couleur, la première l'emporte.
`@UMLDependency` vers une classe déjà atteinte par une association **dessinée en trait**	Une association dit strictement plus qu'une dépendance. Sous `ATTRIBUTE`, rien n'est signalé.
Attribut de type collection sans `targetMultiplicity`, ou multiplicité que le type contredit (`List` pour `"0..1"`)	Aucune multiplicité ne sera portée, ou bien l'un des deux se trompe.
`target` absent du type de l'attribut	Le lien sera tout de même dessiné vers la classe déclarée.
`@UMLDerived`, `@UMLMultiplicity`, `@UMLNote` ou `@UMLConstraint` sur un membre que le diagramme n'affiche pas	Barre oblique, crochets, bulle ou accolades ne se voient que si l'élément marqué est dessiné. `@UMLMultiplicity("")` ne demande rien.
`@UMLDerived` sur un lien **en trait** sans `targetRole`	La barre se pose devant le rôle cible ; sans rôle, rien ne peut la porter.
`@UMLNote` sur un attribut porteur d'une association **en trait**	Une note ne s'accroche pas à un trait.

Chacun de ces contrôles **se tait dès qu'il ne peut pas conclure** : une collection écrite sans paramètre de type (`List` nu) ne se voit jamais reprocher sa cible ; les **composants d'un record** échappent aux contrôles qui disent « ce membre n'est pas affiché » ; et un `@UMLInclude` qui déroge à un `hide` du projet n'est jamais « sans effet ».

17.4 Faire taire les avertissements

L'option `-Aumlg.warnings=none` les éteint tous ; `none` est la seule valeur reconnue, toute autre étant **refusée**, et les erreurs ne s'éteignent pas. Le silence porte sur ce que le processeur a trouvé dans le code, non sur son propre état : un `-Aumlg.rules` désignant un fichier introuvable reste signalé même sous `none`.

17.5 Donner le fichier de règles au compilateur

Six avertissements dépendent de la stratégie appliquée à la classe — ceux qui disent « ce membre n'est pas affiché » ou « il l'est déjà ». Quand la classe la déclare, le processeur conclut seul ; quand elle s'en remet au projet (`PROJECT`, le défaut), la réponse est dans `umlg.json`, que

`javac` ne sait pas trouver. Le processeur **se tait** alors sur ces six contrôles, à moins qu'on ne lui donne le chemin :

```
<compilerArgs>
    <arg>-Aumlg.rules=src/main/resources/umlg.json</arg>
</compilerArgs>
```

Trois situations : **option absente**, le processeur se tait ; **fichier introuvable**, **avertissement** ; **fichier fautif**, **erreur**. Cette option ne change jamais le diagramme produit, et n'ajoute ni ne retire aucune *erreur*.

18 Ce qui est hors périmètre

Écartés de l'API, et à ne pas réintroduire par inadvertance : le **style fin des liens** (couleur, épaisseur, pointillés, longueur) ; la **navigabilité explicite** (`--` contre `-->` contre `<-->`) ; la **pastille de stéréotype** (`«(S,#FF7700) Service»`) et les stéréotypes multiples ; l'**association qualifiée**, que PlantUML ne sait pas dessiner — la donnée qui indexe se dit par le type de l'attribut ; l'**imbrication de paquetages** et les variantes de conteneurs (`namespace`, `folder`, `frame`, `together`) ; les **notes rattachées à un lien** et les notes flottantes ; les **directives brutes** `skinparam` et les thèmes PlantUML ; le **placement d'une boîte par rapport à une autre**, remplacé par l'ordre d'écriture d'une vue (section 16) ; enfin les **boîtes, attributs et liens déclarés hors du code** — une classe encore à écrire s'écrit vide, en Java, avec `@UMLClass`.

18.1 Ce qui relève de l'analyse automatique

Sont déduits du code compilé, sans aucune annotation : la nature de l'entité (classe, classe abstraite, interface, énumération, type annotation, *record*), les liens d'héritage lus dans `extends` et `implements`, les littéraux d'une énumération, le modificateur `static`, la visibilité de chaque membre, et les signatures de méthodes (type de retour, types des paramètres, et leurs noms sous `-parameters`). S'y ajoutent les **paramètres de type** d'une classe générique, écrits à la suite de son nom, bornes comprises — `Boite<T>`, `Table<K, V extends Number>` — mais distincts du nom, qui seul désigne la boîte dans un sélecteur de vue ; et l'italique d'une **opération sans corps**, `abstract` ou déclarée par une interface sans être `default`.

18.2 Ce qui ne peut pas être déduit

La lecture ayant lieu après compilation (section 1.1), les **valeurs d'initialisation des attributs** et le **corps des méthodes** ont disparu — d'où `@UMLDependency`. Reste réellement indéductible la **multiplicité** : rien dans `Set<String>` ne dit `0..*` plutôt que `1..*`. Le **nom des paramètres**, lui, est une option de construction du projet analysé, `-parameters`, qu'aucune annotation ne remplace.

19 Mode d'emploi : de zéro à un dossier de diagrammes

19.1 1. Déclarer la bibliothèque dans le `pom`

Une dépendance — les annotations, et le processeur que `javac` découvre seul dès qu'elle est sur le classpath — et un plugin, qui apporte le goal qui dessine.

```
<properties><maven.compiler.parameters>true</maven.compiler.parameters></
   properties>
<dependency>  <!-- les annotations, et le processeur qu'elles embarquent
   -->
    <groupId>io.github.oliviercailloux</groupId>
    <artifactId>umlg</artifactId> <version>1.0-SNAPSHOT</version>
```

```
</dependency>
<plugin>      <!-- le goal qui dessine -->
    <groupId>io.github.oliviercailloux</groupId>
    <artifactId>umlg-maven-plugin</artifactId> <version>1.0-SNAPSHOT</
   version>
    <executions><execution><goals><goal>generate</goal></goals></execution
   ></executions>
</plugin>
<plugin>      <!-- version epinglee, sinon Maven resout la 3.1 de 2013 -->
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-compiler-plugin</artifactId> <version>3.13.0</version
   >
    <configuration><showWarnings>true</showWarnings></configuration>
</plugin>
```

Les trois réglages du compilateur comptent : sans `parameters`, les diagrammes écrivent `entries(String)` au lieu d'`entries(xKeyName: String)` ; sans `showWarnings`, Maven passe `-nowarn` à `javac` et fait taire le processeur, ses erreurs mises à part ; sans version **épinglée**, Maven résout la 3.1, antérieure à `parameters`, alors ignoré en silence.

19.2 2. Annoter le code

`@UMLClass` sur les classes à dessiner suffit à obtenir un diagramme ; les dix autres sont facultatives et se posent au fur et à mesure qu'on veut en dire plus.

19.3 3. Poser `umlg.json` — à un endroit précis, et à un seul

La bibliothèque cherche le fichier à **un seul chemin**, qu'elle calcule et ne devine pas :

<répertoire analysé>`/umlg.json`

Le **nom est fixe** — exactement `umlg.json`, casse comprise ; la **recherche ne descend pas** dans les sous-dossiers ; et **aucune option ne le déplace** : on change le répertoire analysé, et `-Dumlg.sourceDirectory=autre/chemin` fait attendre le fichier à `autre/chemin/umlg.json`.

Or le répertoire analysé est celui des **classes compilées** — `target/classes` par défaut — et non celui des sources. On écrit donc le fichier là d'où la construction le *recopiera* : `src/main/resources/umlg.json`, dont Maven recopie le contenu à la racine de `target/classes`. Il est ainsi versionné avec le code qu'il décrit. Le fichier est **facultatif** ; *présent mais fautif*, il arrête la génération.

La seule exception. `javac` ne connaît pas `target/classes` : le processeur reçoit le chemin **nommément** par `-Aumlg.rules=...` (section 17.5). Là, et là seulement, le chemin et le nom sont libres.

19.4 4. Lancer

Trois façons, la première étant celle à préférer : **depuis Maven**, `mvn compile umlg:generate` ; **en ligne de commande**, `UMLGenerator` *<répertoire>* `[fichier.svg|dossier] [png|svg]` ; **par programme**, `UMLGenerator.builder()...build().generate()` (section 16.8).

Le `compile` n'est pas facultatif : rien n'est analysable tant que les `.class` ne sont pas écrits, et c'est lui qui recopie `umlg.json` au bon endroit.

Tout ce qui se règle à l'appel. Six propriétés, surchargeables aussi par `<configuration>` dans le `pom` ; `umlg:help` les énumère.

— `umlg.sourceDirectory` (`target/classes`) : le répertoire analysé — et donc où `umlg.json` est cherché ;

— `umlg.outputDirectory` (`diagrammes/`) : le dossier écrit, à la racine du module ;
— `umlg.outputFile` : un seul fichier au lieu du dossier, son extension décidant du format ;
— `umlg.format` (`svg`) : `svg` ou `png`, pour les images du dossier ;
— `umlg.keepPrevious` (`false`) : écrire dans un dossier voisin numéroté plutôt qu'effacer ;
— `umlg.skip` (`false`) : sauter la génération sans arrêter la construction.

Les chemins relatifs s'entendent depuis la racine du module, jamais depuis le répertoire d'où Maven a été lancé. Trois lignes couvrent l'usage courant :

```
mvn compile umlg:generate                                  # le dossier, en SVG
mvn compile umlg:generate -Dumlg.format=png                # le dossier, en PNG
mvn compile umlg:generate -Dumlg.outputFile=archi.svg  # la seule vue d'
   ensemble
```

19.5 5. Ce qu'on obtient

Un dossier `diagrammes/` : la vue intégrale à sa racine, les vues dans `vues/`, la source `.puml` près de chaque image, et `ignored-classes.txt` au-delà d'une douzaine de classes écartées. Compte rendu et avertissements partent dans le log Maven, en `[WARNING]`. Le dossier vit **à la racine du projet**, non dans `target/` : c'est une documentation, faite pour être versionnée.